



Java Web: Servlets, JSP, JSTL e EL.

Desenvolvimento dinâmico no ecossistema Jakarta EE

Objetivos da aula

Ao final desta aula, espera-se compreender como uma aplicação Java Web dinâmica recebe uma **requisição HTTP**, processa regras no **servidor** e devolve uma **resposta HTML**.

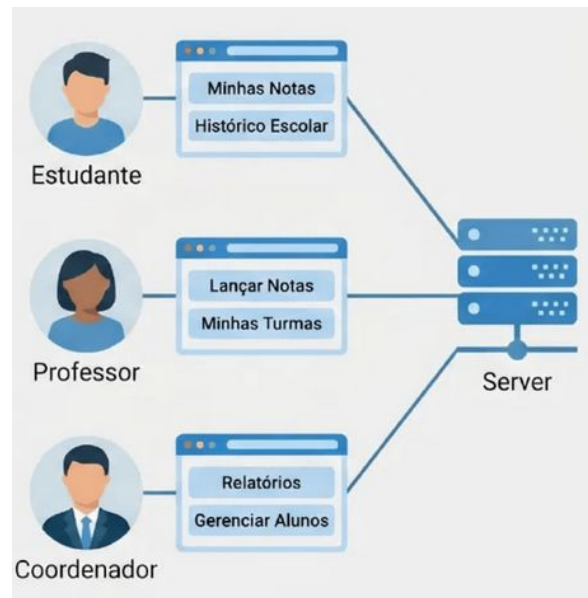
Competência	Resultado esperado
Arquitetura Web	Explicar cliente, servidor, contentor e ciclo HTTP.
Servlets	Criar controladores com HttpServlet, doGet e doPost.
JSP, EL e JSTL	Produzir views limpas, sem lógica Java espalhada.
MVC	Separar responsabilidades entre controller, model e view.

? Problema inicial

Imagine um sistema acadêmico com login, cadastro de alunos, cadastro e consulta de disciplinas, histórico escolar e gestão de diários.

- A **mesma tela pode ser exibida com dados diferentes** para cada estudante, professor ou coordenador.
- A página é resultado de uma **requisição processada no servidor**.

Como transformar **dados, regras de negócio e templates HTML** em **respostas personalizadas** para cada usuário?



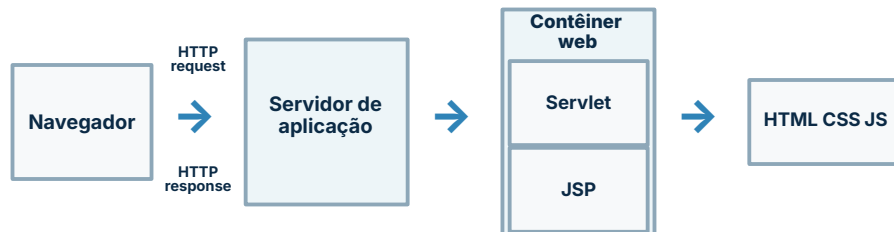
■ Aplicações estáticas e aplicações dinâmicas

Uma página HTML estática é entregue praticamente como foi escrita, enquanto uma aplicação dinâmica monta a resposta no momento da requisição, combinando parâmetros, sessão, dados persistidos e regras de negócio.

Aspecto	HTML estático	Aplicação Java Web dinâmica
Conteúdo	Igual para todos.	Calculado no servidor por requisição.
Regras	Limitadas ao cliente.	Implementadas em Java no servidor.
Estado	Não mantém sessão por padrão.	Usa sessão, cookies, banco e escopos.
Tecnologias	HTML, CSS e JS.	Servlet, JSP, EL, JSTL e serviços.

↔ Arquitetura cliente-servidor

- O navegador envia uma **HTTP request** ao servidor.
- O servidor de aplicação delega a requisição ao **contêiner web**.
- O contêiner executa componentes como **Servlets** e **JSPs**.
- A resposta **HTTP response** retorna HTML, CSS, JavaScript ou dados.



Cliente solicita; servidor processa; contêiner executa; resposta retorna ao navegador.

Serviço web e contêiner Java

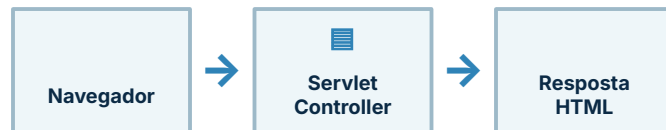
O serviço web recebe chamadas do navegador, executa lógica e responde ao cliente. Em **Jakarta EE** essa lógica é hospedada em um contêiner web ou servidor completo de aplicações como Apache Tomcat, Eclipse Jetty, GlassFish ou WildFly.

Elemento	Função principal
 Navegador	Envia requisições HTTP e renderiza respostas.
 Servidor de aplicação	Hospeda a aplicação e oferece serviços de execução.
 Contêiner web	Gerencia ciclo de vida, concorrência, mapeamento e objetos HTTP.
 Aplicação Web	Empacota classes, JSPs, bibliotecas e descritores.

O que é um Servlet?

- **Definição principal em português:** "Um Servlet é uma classe Java executada no servidor para processar requisições e gerar respostas."
- **Complemento:** "Em aplicações HTTP normalmente estende HttpServlet, que oferece métodos como doGet, doPost, doPut, doDelete, doHead, doOptions e doTrace."

```
@WebServlet("/usuarios")
public class UsuarioServlet extends HttpServlet {
    // controlador HTTP da aplicação
}
```



Em uma arquitetura MVC clássica, o **Servlet frequentemente exerce o papel de Controller.**

Jakarta Servlet 6.1 no contexto atual

Integra **Jakarta EE 11**, exige **Java SE 17** ou superior e define o padrão moderno para desenvolvimento Web em Java.

Java SE 17



Jakarta EE 11



Servlet 6.1

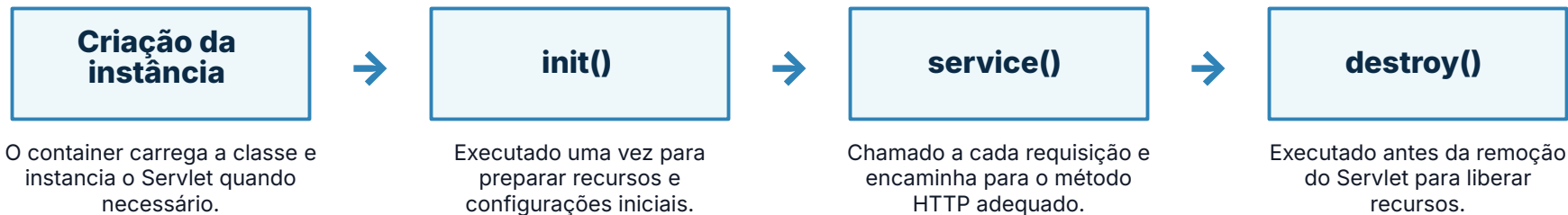
Ponto de atenção	Implicação prática
Pacotes atuais	Usar <code>jakarta.servlet.*</code> , não <code>javax.servlet.*</code> .
Java mínimo	Planejar ambiente com Java 17 ou superior.
API HTTP	Trabalhar com <code>HttpServletRequest</code> e <code>HttpServletResponse</code> .
Contêiner	Executar em servidor compatível com Jakarta Servlet.

Projetos novos:
`jakarta.servlet.*`

Atenção à migração de namespace: `javax.*` → `jakarta.*`

Ciclo de vida do Servlet

O ciclo de vida é **controlado pelo contêiner web**. O programador implementa os métodos, mas não cria nem destrói manualmente as instâncias em cada requisição.

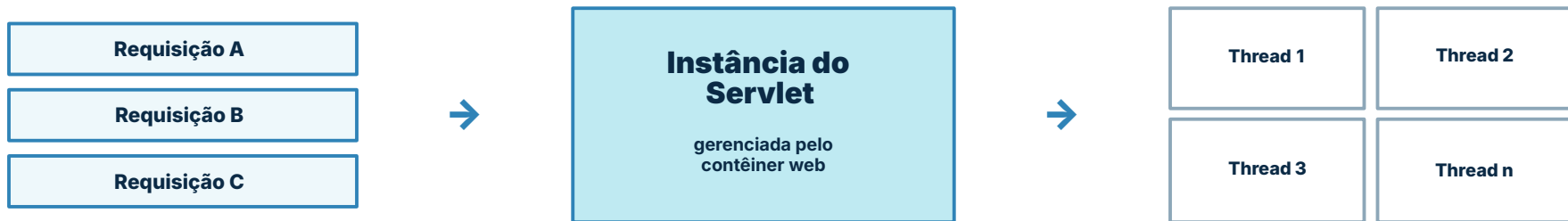


Em `HttpServlet`, o método `service()` despacha para `doGet`, `doPost`, `doPut`, `doDelete` e outros métodos HTTP.

Ideia-chave: o Servlet costuma permanecer carregado e atender múltiplas requisições durante sua vida útil.

≈ Concorrência em Servlets

Um Servlet normalmente possui **uma única instância** mantida pelo contêiner, mas pode atender a **várias requisições simultâneas** por meio de diferentes threads.



Riscos de estado compartilhado

- **Evite** guardar dados da requisição em campos de instância.
- Atributos mutáveis compartilhados podem gerar **condição de corrida**.
- Uma requisição pode sobrescrever dados lidos por outra.

Recomendações práticas

- Use variáveis locais dentro de `doGet` e `doPost`.
- Armazene dados por requisição em `request` ou objetos locais.
- Se precisar compartilhar recursos, aplique sincronização e componentes próprios.

@ Mapeamento com @WebServlet

A anotação **@WebServlet** associa uma URL pública da aplicação a uma classe Java responsável por tratar a requisição HTTP.

- O padrão informado na anotação define o **caminho de acesso** ao Servlet.
- O navegador chama a URL; o contêiner localiza a classe correspondente.
- O mapeamento por anotação reduz configuração XML em aplicações simples.

Exemplo de acesso no navegador

http://localhost:8080/minhaapp/usuarios

/minhaapp
contexto da aplicação

/usuarios
padrão do Servlet

```
@WebServlet("/usuarios")
public class UsuarioServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req,
                          HttpServletResponse resp)
        throws ServletException, IOException {
        // processa a requisição
    }
}
```

URL pública no
navegador



/usuarios



UsuarioServlet

O contêiner web faz a ponte entre a rota HTTP e a classe Java.



Mapeamento com web.xml

Além de anotações como **@WebServlet**, o mapeamento pode ser declarado em um descritor central da aplicação: **WEB-INF/web.xml**.

Configuração centralizada fora da classe Java.

- Declara o **nome lógico** do Servlet.
- Informa a **classe Java** que será executada.
- Associa uma **URL pública** ao Servlet.
- É comum em projetos legados ou cenários com configuração mais explícita.

URL da requisição → **servlet-mapping** → **servlet-class**

Exemplo de WEB-INF/web.xml

```
<web-app>

<servlet>

<servlet-name> UsuarioServlet</servlet-name>

<servlet-class> br.edu.ifce.UsuarioServlet</servlet-class>

</servlet>

<servlet-mapping>

<servlet-name> UsuarioServlet</servlet-name>

<url-pattern> /usuarios</url-pattern>

</servlet-mapping>

</web-app>
```

doGet: consultas e carregamento de páginas

O método **doGet** é chamado quando a requisição HTTP usa o método **GET**, comum em consultas, filtros e abertura de páginas.

- Os parâmetros podem aparecer na **URL**.
- É adequado para operações de **leitura**.
- Não deve ser usado para ações que alteram estado sensível.

Exemplo de chamada:

/buscar?termo=java&pagina=1

</> Exemplo de implementação

```
@Override
protected void doGet(HttpServletRequest request,
                     HttpServletResponse response)
    throws ServletException, IOException {

    String termo = request.getParameter("termo");
    String pagina = request.getParameter("pagina");

    response.setContentType("text/html;charset=UTF-8");

    PrintWriter out = response.getWriter();
    out.println("<h1>Busca por: " + termo + "</h1>");
}
```

Use GET quando a requisição representar consulta, navegação ou carregamento de página.

doPost: envio de formulários

O método **doPost** é chamado quando a requisição HTTP usa **POST**, comum no envio de formulários e em operações que alteram estado.

- Os dados seguem no **corpo da requisição**.
- É usado em cadastro, login, atualização e envio de arquivos.
- Evita expor parâmetros sensíveis diretamente na URL.

Corpo da requisição HTTP

nome=Ana%20Lima
email=ana@ifce.edu.br

Formulário HTML

```
<form action="cadastro" method="post">  
  <input name="nome"> <input name="email">  
</form>
```

Exemplo de implementação

```
@Override  
protected void doPost(HttpServletRequest request,  
                      HttpServletResponse response)  
    throws ServletException, IOException {  
  
    request.setCharacterEncoding("UTF-8");  
  
    String nome = request.getParameter("nome");  
    String email = request.getParameter("email");  
  
    // processa, valida e salva os dados  
    response.sendRedirect("sucesso.jsp");  
}
```

Use POST quando a requisição envia dados e representa criação, autenticação ou alteração de estado.



GET versus POST

A escolha entre **GET** e **POST** deve considerar onde os dados trafegam, qual ação a requisição representa e se a operação pode ser repetida com segurança.

Critério	GET	POST
Local dos dados	Parâmetros aparecem na URL, após o caractere <code>?</code> .	Dados seguem no corpo da requisição HTTP.
Uso típico	Consultas, filtros, buscas e carregamento de páginas.	Envio de formulários, login, cadastro e alterações.
Repetição	Normalmente pode ser repetido ou atualizado sem alterar estado.	Pode causar nova gravação ou nova ação no servidor.
Segurança percebida	Não usar para senha ou dados sensíveis visíveis na URL.	Ocultos os dados da URL, mas ainda exige HTTPS e validação.
Semântica	Ler ou recuperar representação de recurso.	Submeter dados para processamento no servidor.

GET : consultar, navegar e recuperar informações.

POST : enviar dados e solicitar processamento.

■ Formulário HTML e parâmetros

Em formulários HTML, o atributo **name** de cada campo define a chave enviada na requisição e recuperada no Servlet com **request.getParameter(...)** .

View: formulário HTML

```
<form method="post" action="cadastro" >

  <label> Nome</label>
  <input type="text" name="nome" >

  <label> E-mail</label>
  <input type="email" name="email" >

  <label> Curso</label>
  <input type="text" name="curso" >

  <button type="submit" >Enviar</button>

</form>
```



requisição
HTTP

name
vira parâmetro

Controller: leitura no Servlet

```
@Override
protected void doPost(HttpServletRequest request,
                        HttpServletResponse response)
    throws ServletException, IOException {

    String nome = request.getParameter("nome");
    String email = request.getParameter("email");
    String curso = request.getParameter("curso");

    // os valores podem ser validados,
    // convertidos e enviados para a view
}
```

Regra prática: `<input name="nome">` é recuperado por `request.getParameter("nome")` .



Capturando dados no Servlet

Parâmetros enviados pelo formulário chegam como **String**. Antes de usar esses dados, o Servlet deve capturar, converter, validar e tratar possíveis erros de entrada.

1**Captura**

Usar `request.getParameter("nome")`

2**Converter**

transformar texto em número, data ou booleano

3**Validar**

verificar campos obrigatórios e faixas aceitáveis

4**Tratar erro**

devolver mensagem clara para a view

Nunca confie cegamente nos dados recebidos do cliente.

Exemplo: leitura, conversão e validação

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws ServletException, IOException {

    String nome = request.getParameter("nome");
    String idadeTexto = request.getParameter("idade");

    try {
        int idade = Integer.parseInt(idadeTexto);

        if (nome == null || nome.isBlank() || idade <= 0) {
            request.setAttribute("erro", "Dados inválidos.");
            request.getRequestDispatcher("/form.jsp")
                .forward(request, response);
            return;
        }

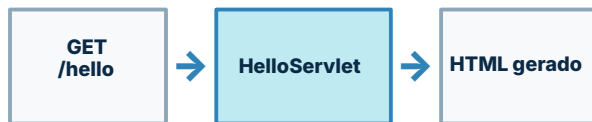
        request.setAttribute("nome", nome);
        request.setAttribute("idade", idade);
        request.getRequestDispatcher("/resultado.jsp")
            .forward(request, response);

    } catch (NumberFormatException e) {
        request.setAttribute("erro", "Idade deve ser numérica.");
        request.getRequestDispatcher("/form.jsp")
            .forward(request, response);
    }
}
```

{ } Exemplo mínimo: Hello Servlet

Um Servlet mínimo pode responder diretamente ao navegador no método **doGet**, definindo o tipo da resposta e escrevendo HTML no corpo da **HTTP response**.

- **@WebServlet** define a rota pública.
- **HttpServlet** fornece a base para tratar HTTP.
- **doGet** processa a requisição GET.
- **PrintWriter** escreve a resposta enviada ao cliente.



Este exemplo é didático: em projetos organizados, a geração de HTML tende a ficar em JSPs ou outra template engine.

HelloServlet.java

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    @Override

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html; charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h1>Olá, Servlet!</h1>");
    }
}
```

Requisição GET em /hello → execução do Servlet → resposta HTML exibida no navegador.

! O problema de gerar HTML no Servlet

Embora seja possível escrever HTML diretamente no Servlet, misturar **controle HTTP**, **regra de negócio** e **marcação visual** torna o código difícil de manter.

HTML impresso dentro do Servlet

```
protected void doGet(HttpServletRequest req,
                    HttpServletResponse resp)
    throws ServletException, IOException {

    resp.setContentType("text/html;charset=UTF-8");
    PrintWriter out = resp.getWriter();

    out.println("<html>");
    out.println("<head><title>Usuários</title></head>");
    out.println("<body>");
    out.println("<h1>Lista de usuários</h1>");
    out.println("<p>Total: " + total + "</p>");
    out.println("</body></html>");
}
```

Problema: qualquer alteração visual exige editar, recompilar e testar código Java.

Separação recomendada de responsabilidades

Servlet Controller

Recebe a requisição, valida entrada, coordena o fluxo e encaminha para a view.

Model Regras /

Representa dados, executa regras de negócio e interage com serviços ou persistência.

JSP View

Apresenta HTML com EL/JSTL, sem espalhar lógica Java pela página.

O Servlet prepara dados; a JSP transforma esses dados em interface HTML.

➡ Redirecionamento com sendRedirect

O método **sendRedirect** instrui o navegador a acessar outra URL. O servidor responde com um redirecionamento HTTP, e o cliente inicia uma **nova requisição**.

Fluxo do redirecionamento



A URL visível no navegador muda para o destino informado no redirecionamento.

Exemplo de uso no Servlet

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws ServletException, IOException {

    // após salvar ou autenticar
    response.sendRedirect("sucesso.jsp");
}
```

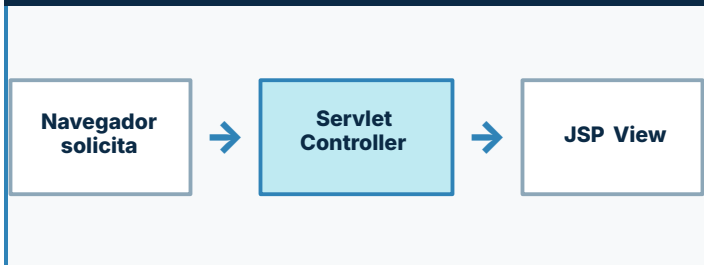
Características importantes

- Cria uma **nova requisição** HTTP no navegador.
- Atributos em **request** não são preservados.
- Pode apontar para URL interna ou externa.
- É comum após cadastro, login, logout ou operações concluídas.

↪ Encaminhamento com forward

O **forward** encaminha a requisição internamente para outro recurso do servidor, preservando os objetos **request** e **response** e os atributos definidos pelo Servlet.

Fluxo interno no servidor



A mesma request carrega atributos do controller para a JSP.

- A URL do navegador **não muda** .
- O processamento permanece **no servidor** .
- É útil para aplicar o padrão **Servlet → JSP** .

Exemplo com RequestDispatcher

```
Usuario usuario = buscarUsuario(request.getParameter("id"));
request.setAttribute("usuario", usuario);

RequestDispatcher dispatcher =
    request.getRequestDispatcher("/resultado.jsp");

dispatcher.forward(request, response);
```

Característica	Com forward
Requisição	A mesma requisição continua sendo usada.
Atributos	São preservados e acessíveis na JSP.
URL no navegador	Permanece a URL chamada inicialmente.
Uso típico	Enviar dados do Servlet controller para a view.

forward é despacho interno: o navegador não faz uma nova requisição.

+ Inclusão com include

A operação **include** permite compor a resposta atual adicionando a saída produzida por outro recurso, sem transferir completamente o controle da requisição.

Resposta composta no servidor

/WEB-INF/partes/cabecalho.jsp

/WEB-INF/partes/menu.jsp

Conteúdo principal da página atual

/WEB-INF/partes/rodape.jsp

Vários recursos contribuem para uma única resposta HTTP enviada ao navegador.

Exemplo com RequestDispatcher.include

```
protected void doGet(HttpServletRequest request,
                    HttpServletResponse response)
    throws ServletException, IOException {

    response.setContentType("text/html;charset=UTF-8");

    RequestDispatcher cabecalho =
        request.getRequestDispatcher("/WEB-INF/partes/cabecalho.jsp");
    cabecalho.include(request, response);
    // escreve ou encaminha o conteúdo principal
    RequestDispatcher rodape =
        request.getRequestDispatcher("/WEB-INF/partes/rodape.jsp");
    rodape.include(request, response);
}
```

Usos comuns

- Reutilizar **cabeçalho**, **menu** e **rodapé**.
- Montar trechos compartilhados sem duplicar HTML.
- Compor a resposta mantendo a requisição atual.

↔ Redirect, forward e include em perspectiva

As três técnicas controlam o fluxo entre recursos Web, mas diferem no número de requisições, na URL exibida ao usuário, na participação do navegador e na forma como a resposta é construída.

Critério	sendRedirect	forward	include
Requisição	Cria uma nova requisição HTTP.	Reaproveita a mesma requisição .	Usa a mesma requisição durante a composição.
URL no navegador	Muda para a nova URL enviada ao cliente.	Permanece a URL original da requisição.	Permanece a URL do recurso principal.
Servidor e navegador	O navegador participa seguindo a resposta 302.	O redirecionamento é interno ao servidor.	O servidor inclui a saída de outro recurso.
Resposta	A resposta final vem da nova URL chamada.	A resposta é produzida pelo recurso encaminhado.	A resposta é a soma do principal com o incluído.
Uso principal	Navegar para outra página, pós-cadastro ou pós-login.	Enviar dados do Servlet para uma JSP de resultado.	Reutilizar cabeçalho, menu, rodapé ou trechos comuns.

redirect: muda a rota visível e inicia nova requisição.

forward: transfere internamente para outra view.

include: compõe partes dentro da mesma resposta.

⇒ Atributos de request

Na mesma requisição, o Servlet pode receber **parâmetros enviados pelo cliente** e também criar **atributos no servidor** para compartilhar objetos com a JSP.



Parâmetros do cliente

São valores enviados pelo navegador por formulário ou query string; chegam ao Servlet como **String**.

```
request.getParameter("termo")
```

```
/buscar?termo=java
```

Origem: cliente HTTP.

Atributos do servidor

São objetos colocados pelo Servlet na request para serem consumidos por outro recurso, como uma **JSP**.

```
request.setAttribute("usuario", usuario)
```

```
${usuario.nome}
```

Origem: código executado no servidor.

Padrão recomendado: o controller prepara os dados e a JSP apenas apresenta esses atributos.



Escopos e Expression Language

A **Expression Language** permite acessar dados disponíveis nos escopos da aplicação usando a sintaxe **`${...}`**, reduzindo código Java na JSP.

Escopo	Duração	Uso típico
pageScope	Apenas a página JSP atual.	Valores locais usados somente durante a renderização da página.
requestScope	Enquanto a requisição estiver ativa.	Dados preparados pelo Servlet e enviados para a JSP com forward .
sessionScope	Durante a sessão do usuário.	Informações associadas ao usuário, como login e preferências.
applicationScope	Enquanto a aplicação estiver carregada.	Dados compartilhados por toda a aplicação, visíveis a múltiplos usuários.

Resolução de nomes em EL

<code>\${usuario}</code>	busca automaticamente o atributo nos escopos disponíveis.
<code>page</code>	primeiro procura na página atual.
<code>request</code>	depois verifica a requisição encaminhada.
<code>session/app</code>	por fim consulta sessão e aplicação.

```
${requestScope.usuario.nome}  
${sessionScope.usuarioLogado.email}  
${param.termo}  
${applicationScope.nomeSistema}
```

Use prefixos como `requestScope` quando houver ambiguidade entre nomes.

JS P Introdução ao JSP / Jakarta Pages

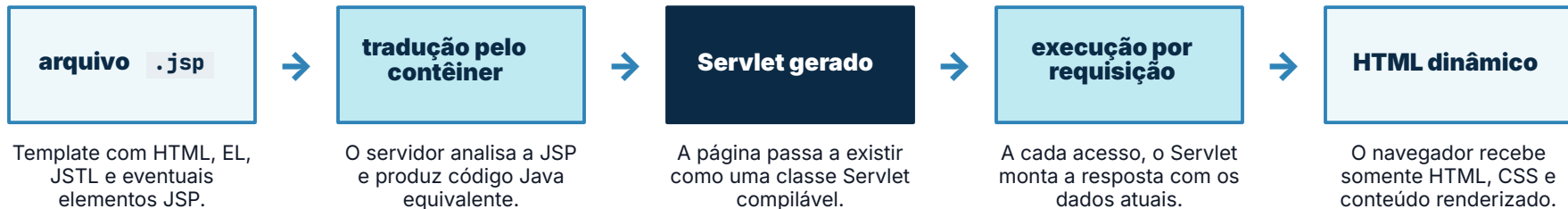
JSP, atualmente associada ao nome **Jakarta Pages**, é uma tecnologia de **template para views**: permite escrever páginas HTML/XML com pontos dinâmicos preenchidos no servidor.



Objetivo didático: Servlet controla o fluxo; JSP apresenta os dados em HTML.

⇒ O que acontece internamente com uma JSP?

Uma página **JSP** não é executada como um arquivo HTML comum: o container a transforma em um **Servlet Java**, que então gera a resposta dinâmica para o navegador.



Primeiro acesso e alterações

Na primeira requisição, ou após mudança no arquivo, o container pode **traduzir e compilar** a JSP antes de atender a página.

Dois momentos do processamento

Tradução

`pagina.jsp` vira Servlet Java.

Execução

O Servlet processa a request e gera HTML.

Ideia-chave: JSP é uma forma de escrever a view, mas internamente ela é convertida em Servlet pelo container.

Benefícios e responsabilidades de JSP

JSP é mais útil quando funciona como **camada de apresentação** : recebe dados preparados pelo Servlet, renderiza HTML e evita concentrar regras de negócio na view.

Benefício	Consequência didática	Responsabilidade prática
Template natural	Permite escrever HTML próximo ao formato final visto pelo navegador.	Manter a página focada em estrutura visual, textos, formulários e marcação.
EL e JSTL	Reduz a necessidade de Java embutido na página.	Exibir atributos, listas e decisões simples de apresentação com tags e expressões.
Separação de papéis	Ajuda a distinguir controller, model e view em aplicações Java Web.	Receber dados já preparados pelo Servlet e não executar regra de negócio complexa.
Reuso por tags	Evita repetição de trechos comuns e torna a view mais organizada.	Usar tags, inclusões e fragmentos para cabeçalho, menu, rodapé e componentes simples.
Manutenção da interface	Alterações visuais ficam menos dependentes de recompilação de classes Java.	Preservar a legibilidade do HTML e manter lógica pesada fora da JSP.

Elementos JSP clássicos

A JSP clássica possui marcações próprias para configurar a página e inserir conteúdo dinâmico. Elas ajudam a entender a tecnologia, mas o desenvolvimento moderno privilegia **EL** e **JSTL**.

Diretiva	configuração
<pre><%@ page contentType="text/html; charset=UTF-8" %></pre>	
Define instruções para a tradução da página , como encoding, imports e taglibs.	
Usada para configurar a JSP antes da execução.	

Scriptlet	lógica
<pre><% if (logado) { %> Bem-vindo! <% } %></pre>	
Permite inserir comandos Java dentro do corpo da JSP.	
Deve ser evitado: mistura regra e apresentação.	

Expressão	saída
<pre><%= usuario.getNome() %></pre>	
Avalia uma expressão Java e escreve o resultado diretamente na resposta HTML.	
Hoje costuma ser substituída por <code>\${usuario.nome}</code> . EL	

Declaração	membro
<pre><%! private int contador = 0; %></pre>	
Declara campos ou métodos que entram na classe Servlet gerada.	
Requer cuidado com concorrência e estado compartilhado.	

Síntese: elementos clássicos explicam como JSP funciona, mas views mais limpas usam HTML + EL + JSTL, mantendo Java fora da página.

<%@ Diretivas JSP

As **diretivas JSP** configuram a página durante a fase de tradução, antes do processamento das requisições. Elas orientam o container sobre imports, inclusão estática e bibliotecas de tags.

page

Configura propriedades da página JSP, como **linguagem, linguagem**, imports e tratamento de erros.

Exemplo:

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
```

Define como a JSP será traduzida e como a resposta será configurada.

include

Inclui estaticamente outro arquivo no momento da **tradução** da JSP, antes da execução da requisição.

Exemplo:

```
<%@ include file="/WEB-INF/partes/menu.jsp"%>
```

Útil para fragmentos compartilhados, como cabeçalho, menu e rodapé.

taglib

Registra uma biblioteca de tags e define um **prefixo** para usar tags customizadas, como JSTL.

Exemplo:

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
```

Após a declaração, a página pode usar tags como `<c:forEach>`.

Ideia-chave

Diretivas não produzem conteúdo visual diretamente; elas configuram a tradução da JSP e preparam o ambiente da página.

Scriptlets: por que evitar?

Scriptlets permitem inserir **código Java diretamente na JSP** , mas essa prática mistura apresentação com regra de negócio e enfraquece a separação de responsabilidades.

JSP com scriptlet: Java misturado ao HTML

```
<%  
    List<Usuario> usuarios =  
        (List<Usuario> ) request.getAttribute("usuarios");  
  
    for (Usuario u : usuarios) {  
%>  
<li>  
    <%= u.getNome() %> - <%= u.getEmail() %>  
</li>  
<%  
    }  
%>  
  
// HTML, iteração e acesso a objetos ficam acoplados.
```

Problema: a página passa a conter lógica Java, ficando mais difícil de ler, testar e manter.

Recomendação didática

1

Mova regras para Java

Use Servlet, services, DAOs ou classes de domínio para processar dados.

2

Exponha dados como atributos

```
request.setAttribute("usuarios", usuarios)
```

3

Renderize com EL/JSTL e HTML

Use `${usuario.nome}` e tags como `<c:forEach>` .

A JSP fica responsável pela apresentação; o Java fica responsável por regras e fluxo.

`${ }` Expression Language: sintaxe essencial

A **Expression Language** usa expressões declarativas para recuperar valores, acessar propriedades, ler parâmetros e avaliar operações simples dentro da JSP.

`${ ... }`

Tudo que está entre chaves é avaliado pelo contêiner no servidor e substituído pelo resultado textual na resposta HTML.

Acesso a propriedades

`${usuario.nome}`

`${usuario.email}`

Chama getters por convenção:
`getNome()`, `getEmail()`.

Parâmetros da requisição

`${param.termo}`

`${paramValues.opcao[0]}`

Útil para ler valores enviados por query string ou formulário.

Operadores

`${idade ≥ 18}`

`${not empty lista}`

Aceita operadores relacionais, lógicos, aritméticos e verificação de vazio.

Literais e expressões

`${'ADS'} · ${10 + 2}`

`${usuario.ativo ? 'Sim' : 'Não'}`

Permite combinar valores simples e expressões condicionais.

Exemplos em uma JSP

`<p>Nome: ${usuario.nome}</p>`

`<p>Busca: ${param.termo}</p>`

`<p>Maior de idade: ${usuario.idade ≥ 18}</p>`

`<p>Lista vazia? ${empty usuarios}</p>`

`<p>Curso: ${requestScope.curso.nome}</p>`

Quando houver nomes repetidos em escopos diferentes, prefira explicitar o escopo: `requestScope`, `sessionScope` ou `applicationScope`.

EL antes e depois

A **Expression Language** substitui acesso explícito e verboso a objetos Java por expressões declarativas, tornando a JSP mais legível e próxima do HTML.

Antes: acesso explícito em Java

```
<%  
  
Usuario usuario = (Usuario)  
    request.getAttribute("usuario");  
%>  
  
<p>Nome: <%= usuario.getNome() %></p>  
<p>E-mail: <%= usuario.getEmail() %></p>  
<p>Curso: <%= usuario.getCurso() %></p>
```

A página mistura HTML com Java, exige cast explícito e espalha chamadas de getter pela JSP.



**menos
verbosidade**

**mesmo dado,
sintaxe mais
declarativa**

Depois: EL declarativa

```
<p>Nome: ${usuario.nome} </p>  
<p>E-mail: ${usuario.email} </p>  
<p>Curso: ${usuario.curso} </p>  
  
// A EL procura o atributo "usuario"  
// e acessa propriedades por convenção:  
// getNome(), getEmail(), getCurso()
```

A view fica declarativa: acessa propriedades do atributo sem código Java explícito. As propriedades seguem a convenção de getters.

JSTL: biblioteca padrão de tags.

A **Jakarta Standard Tag Library** reúne tags reutilizáveis para JSP, permitindo escrever views mais declarativas, com menos scriptlets e melhor separação entre apresentação e código Java.

JSTL

biblioteca padrão de tags para JSP

Fornecer comandos declarativos para saída, decisões, iteração, formatação e funções sobre textos e coleções.

Core

c

`jakarta.tags.core`

Controle básico da view: saída, variáveis, decisões e iteração. Exemplos: `<c:out>`, `<c:if>`, `<c:forEach>`.

Formatting

fmt

`jakarta.tags.fmt`

Formatação e internacionalização: números, datas, mensagens e localização. Exemplos: `<fmt:formatDate>`, `<fmt:formatNumber>`.

Functions

fn

`jakarta.tags.functions`

Funções para EL: operações úteis sobre strings e coleções. Exemplos: `fn:length(...)`, `fn:contains(...)`, `fn:toUpperCase(...)`.

XML / SQL

x · sql

`jakarta.tags.xml · jakarta.tags.sql`

Tags especializadas: processamento XML e consultas SQL. *Em aplicações organizadas, acesso a dados costuma ficar fora da JSP.*

Síntese: use JSTL + EL para views declarativas; deixe regras de negócio e persistência em classes Java apropriadas.

taglib Importando JSTL moderna

Em projetos baseados em **Jakarta EE**, as tags JSTL modernas são importadas na JSP por diretivas **taglib**, associando uma URI da biblioteca a um prefixo usado na página.

Diretivas taglib em uma JSP

```
<!-- Jakarta Tags / JSTL moderna -->
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>

<%@ taglib prefix="fn" uri="jakarta.tags.functions" %>
```

Prefixos e usos frequentes

Prefixo	URI	Uso típico
c	jakarta.tags.core	condições, iteração, saída e controle simples de fluxo.
fmt	jakarta.tags.fmt	formatação de datas, números e mensagens localizadas.
fn	jakarta.tags.functions	funções utilitárias para strings e coleções em EL.

c:out saída segura

A tag **<c:out>** imprime valores na JSP aplicando escape XML/HTML por padrão, ajudando a impedir que texto fornecido pelo usuário seja interpretado como marcação ou script.

Saída direta com EL

```
<p>${comentario} </p>
```

A expressão **\${...}** é curta e legível, mas a política de escape depende do contexto e da configuração do ambiente.

Cuidado ao exibir conteúdo vindo de formulários, URL, banco de dados ou usuários externos.

Escape transforma marcação em texto

ENTRADA RECEBIDA

```
<script>alert('xss')</script>
```



SAÍDA ESCAPADA

```
&lt;script&gt;alert('xss')&lt;/script&g  
t;
```

O navegador passa a exibir o texto, em vez de executar a marcação como código.

Saída com JSTL

```
<c:out value="${comentario}" />  
<c:out value="${comentario}"  
escapeXml="true" />
```

c:out escreve o valor com escape XML/HTML, reduzindo risco de injeção de tags e scripts.

Use especialmente quando o dado não foi controlado pela própria aplicação.

<c:if> Decisão na view.

A tag **<c:if>** permite renderizar trechos da JSP apenas quando uma condição de apresentação for verdadeira, usando expressões booleanas da **Expression Language**.

Forma geral da tag

```
<c:if test="${condicao}" >
    conteúdo exibido
</c:if>
```

Como ler o atributo test

Se a expressão em **\${...}** resultar em verdadeiro, o corpo da tag entra no HTML final.

Use para decisão de renderização visual simples, não para regra de negócio.

Exemplo em JSP com JSTL core

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<c:if test="${not empty erro}" >
    <p class="mensagem-erro">${erro}</p>
</c:if>

<c:if test="${usuarioLogado}" >
    <p>Bem-vindo, ${sessionScope.usuario.nome}!</p>
</c:if>

<c:if test="${empty usuarios}" >
    <p>Nenhum usuário cadastrado.</p>
</c:if>
```

A condição deve responder a uma pergunta de apresentação: "exibir ou não este trecho?".

<c:forEach: iteração de coleções

A tag **c:forEach** percorre coleções disponibilizadas pelo Servlet e permite renderizar listas na JSP usando **EL**, sem escrever laços Java com scriptlets.



Controller: dados no request

```
protected void doGet(HttpServletRequest request,
                    HttpServletResponse response)
    throws ServletException, IOException {

    List<Usuario> usuarios = usuarioService.listar();

    request.setAttribute("usuarios", usuarios);

    request.getRequestDispatcher("/usuarios.jsp")
        .forward(request, response);
}
```

O Servlet prepara a coleção e encaminha a mesma requisição para a view.

View: iteração com c:forEach

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

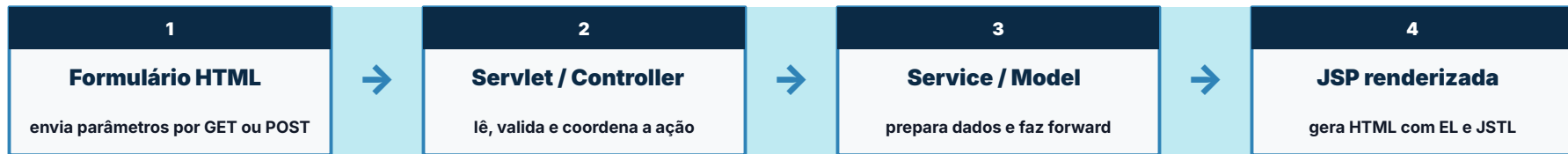
<ul>
  <c:forEach var="usuario" items="${usuarios}">
    <li>${usuario.nome} - ${usuario.email} </li>
  </c:forEach>
</ul>
```

var nomeia o item atual; **items** recebe a coleção a percorrer.

MVC Fluxo Formulário → Servlet → Service/Model → JSP.

Fluxo recomendado para separar responsabilidades: o **formulário** coleta dados, o **Servlet** controla a requisição e a **JSP** renderiza a resposta com EL/JSTL.

Podemos inserir ainda um componente adicional, **relativo à camada model**, para tratar das regras de negócio e acesso a dados, a ser chamado no controller, antes de enviar os dados e resultado para a JSP.



Controller: processa e encaminha

```
protected void doGet(HttpServletRequest request,
    HttpServletResponse response) throws ServletException,
    IOException {
    String nome = request.getParameter("nome");
    List<Usuario> usuarios = usuarioService.buscarPeloNome();

    request.setAttribute("usuarios", usuarios);
    request.getRequestDispatcher("/WEB-INF/views/retorno.jsp")
        .forward(request, response);
}
```

forward mantém a mesma requisição e permite que a JSP leia os atributos preparados.

View: apresenta o resultado

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<ul>
  <c:forEach var="usuario" items="${usuarios}">
    <li>${usuario.nome} - ${usuario.email} </li>
  </c:forEach>
</ul>
```

A JSP não controla o fluxo: apenas transforma atributos em HTML final.

for m Exemplo completo: formulário de cadastro

View de entrada : um formulário HTML coleta nome, e-mail e curso, enviando os dados por **POST** para o Servlet responsável pelo processamento.

View: formulário apresentado ao usuário

Cadastro de usuário

Nome

E-mail

Curso

Enviar

A view coleta dados e define o destino da requisição; ela não valida regra de negócio nem cria o objeto final.

HTML do formulário

```
<form action="${pageContext.request.contextPath}/cadastro" method="post">

  <label for="nome">Nome</label>
  <input id="nome" name="nome" type="text">

  <label for="email">E-mail</label>
  <input id="email" name="email" type="email">

  <label for="curso">Curso</label>
  <select id="curso" name="curso">
    <option>ADS</option>
  </select>

  <button type="submit">Enviar</button>
</form>
```

name define o nome do parâmetro lido no Servlet;

method="post" indica envio de dados para processamento.

POST Exemplo completo: Controller Servlet

O **Controller Servlet** recebe a submissão do formulário, lê os parâmetros enviados por POST, cria o objeto de domínio e encaminha a requisição para a JSP responsável pela resposta visual.

CadastroServlet.java

```
@WebServlet("/cadastro")
public class CadastroServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");

        String nome = request.getParameter("nome");
        String email = request.getParameter("email");
        String curso = request.getParameter("curso");

        Usuario usuarioCriado = usuarioService.criar(new Usuario(nome, email, curso));

        request.setAttribute("usuario", usuarioCriado);
        request.setAttribute("mensagem", "Cadastro realizado!");

        request.getRequestDispatcher("/WEB-INF/views/resultado.jsp")
            .forward(request, response);
    }
}
```

Responsabilidades do Controller

1

Receber POST

Mapeamento em `/cadastro` concentra o processamento do formulário.

2

Ler parâmetros

`getParameter` recupera nome, e-mail e curso enviados pela view.

3

Montar o Model

O objeto `UsuarioService` recebe os dados e cadastra o novo usuário.

4

Encaminhar

`setAttribute` + `forward` enviam dados para a página de retorno.

Bean Exemplo completo: JavaBean Usuario

O **JavaBean Usuario** representa os dados da entidade manipulados no Model. A JSP acessa suas propriedades com **Expression Language**, seguindo a convenção dos getters.

Classe de modelo: Usuario.java

```
package br.edu.ifce.web.entities;

public class Usuario {

    private Long id;
    private String nome;
    private String email;
    private String curso;

    public Usuario(String nome, String email, String curso) {
        this.nome = nome;
        this.email = email;
        this.curso = curso;
    }

    public Long getId() {
        return id;
    }
    ...
}
```

Convenção JavaBean → EL

Método público	Acesso na JSP
getNome()	<code>\${usuario.nome}</code>
getEmail()	<code>\${usuario.email}</code>
getCurso()	<code>\${usuario.curso}</code>

Model Exemplo completo: UsuarioService

A classe **UsuarioService** implementa o caso de uso de cadastro de usuário, manipulando e representando os dados através da entidade **Usuario**.

Classe de modelo: UsuarioService.java

```
package br.edu.ifce.web.services;

import br.edu.ifce.web.entities.Usuario;

public class UsuarioService {

    private static final List< Usuario> usuarios = new ArrayList<>();
    private static final AtomicLong CONTADOR_ID = new AtomicLong(1);

    public UsuarioService () {}

    public Usuario criar(Usuario novoUsuario) {
        Long novoId = CONTADOR_ID.getAndIncrement();

        Usuario usuarioCriado = new Usuario(novoUsuario.getNome(),
                                              novoUsuario.getEmail(),novoUsuario.getCurso());
        usuarioCriado.setId(novoId);

        usuarios.add(usuarioCriado);
        return usuarioCriado;
    }
    ...
}
```

Convenção JavaBean → EL

Método público	Acesso na JSP
getNome()	<code>\${usuario.nome}</code>
getEmail()	<code>\${usuario.email}</code>
getCurso()	<code>\${usuario.curso}</code>

JSP Exemplo completo: JSP de retorno

A **JSP de retorno** recebe atributos preparados pelo Controller Servlet e usa **EL/JSTL** para exibir os dados do cadastro, sem executar regra de negócio na página.

retorno.jsp: apresentação dos dados cadastrados

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<h1>Cadastro realizado</h1>
<p>
  Olá, <c:out value="${usuario.nome}" />!
</p>

<ul>
  <li>E-mail: <c:out value="${usuario.email}" /></li>
  <li>Curso: <c:out value="${usuario.curso}" /></li>
</ul>
```

←!— A JSP apenas renderiza o atributo "usuario". →

O objeto `usuario` foi colocado no request pelo Servlet antes do forward para esta página.

Resultado renderizado no navegador

Cadastro realizado

Nome	Ana Lima
E-mail	ana@ifce.edu.br
Curso	ADS

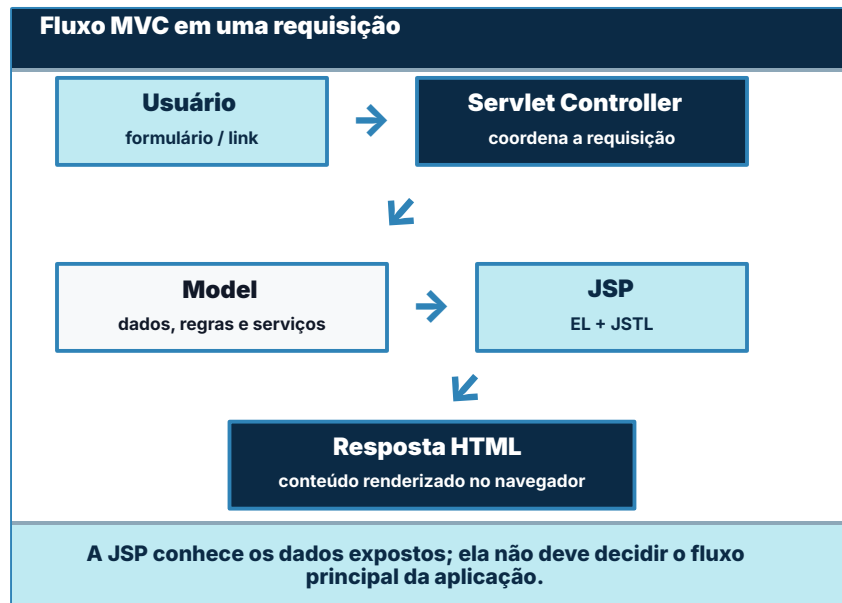
**View
limpa:**

a página apresenta uma confirmação e lê propriedades por convenção de getters, como `getNome()`

MVC MVC com Servlet, Model e JSP

O padrão **MVC** organiza a aplicação Java Web em responsabilidades distintas: o **Servlet** controla a requisição, o **Model** representa regras e dados, e a **JSP** apresenta a resposta.

Camadas e responsabilidades		
Camada	No Java Web	Responsabilidade
Controller	Servlet	Recebe a requisição, lê parâmetros, valida fluxo, chama serviços e encaminha para a view.
Model	JavaBean, Service, DAO	Representa dados, regras de domínio e operações necessárias para produzir a resposta.
View	JSP + EL/JSTL	Renderiza HTML com dados preparados, evitando regra de negócio e código Java na página.
Separar camadas facilita manutenção, teste, leitura do código e evolução da aplicação.		



! Antipadrões comuns

Em aplicações Java Web, alguns atalhos funcionam nos primeiros testes, mas comprometem **manutenção** , **segurança** , **concorrência** e **separação de responsabilidades**.

Antipadrão	Por que é problema?	Prática recomendada
1 SQL direto na JSP <code><% consulta SQL %></code>	Mistura acesso a dados com apresentação, dificulta testes e expõe a página a decisões que pertencem ao backend.	Mover acesso ao banco para DAO/Repository/Service; a JSP apenas recebe atributos prontos.
2 Scriptlets extensos <code><% for (...) { ... } %></code>	Transforma a JSP em código Java difícil de ler, manter e revisar, quebrando a lógica de view declarativa.	Substituir por EL/JSTL, como <code>\${...}</code> , <code><c:if></code> e <code><c:forEach></code> .
3 Dados de request no campo do Servlet <code>private Usuario usuario;</code>	Servlets são compartilhados entre requisições; campos mutáveis podem causar vazamento de dados e erros de concorrência.	Usar variáveis locais e <code>request.setAttribute</code> para dados específicos de cada requisição.
4 Redirect esperando atributos <code>sendRedirect(...)</code>	Redirect cria nova requisição; atributos definidos no request anterior não estarão disponíveis na JSP de destino.	Usar <code>forward</code> quando precisar manter atributos de request, ou guardar mensagem em escopo adequado.

HTTP Atividade rápida 1: leitura de requisição

Observe a requisição abaixo e identifique seus componentes. O objetivo é praticar a leitura de **método HTTP**, caminho, parâmetros e adequação da URL.

URL para
análise

GET /alunos/buscar?termo=ana&pagina=2

Decompondo a requisição

Método	
Caminho	
Query string	
Parâmetro 1	
Parâmetro 2	
Operação	

Perguntas para responder

1	Qual é o método HTTP usado nesta requisição?
2	Qual é o termo pesquisado enviado para o servidor?
3	Qual página da listagem está sendo solicitada?
4	Essa URL é adequada para busca? Justifique pelo método e pelos parâmetros.

GET? Atividade rápida 2: escolha entre GET e POST

Para cada operação, decida o **método HTTP mais adequado** e justifique a escolha considerando leitura, alteração de estado, exposição de parâmetros e sensibilidade dos dados.

Complete a tabela em dupla

Operação	Método	Justificativa curta
Buscar alunos por nome ou matrícula		
Cadastrar novo aluno		
Abrir página de edição		
Salvar edição de cadastro		
Enviar login e senha		

Após preencher, compare as respostas com outra dupla e discuta os casos em que a escolha pode depender do contexto.

Critérios de decisão

GET

Use quando a operação for de leitura.

Exemplos: abrir página, filtrar, pesquisar, listar e consultar informações.

POST

Use quando houver envio para processamento.

Exemplos: cadastrar, salvar, autenticar e executar ação que altera estado.

Atenção

Método HTTP não substitui validação, autenticação, autorização nem uso de **HTTPS**.

IMC Atividade prática 1: IMC com Servlet e JSP

Implemente uma aplicação Java Web simples para calcular o **Índice de Massa Corporal**, praticando formulário HTML, Servlet Controller, atributos de requisição e JSP de resultado.

Enunciado da atividade

Crie uma página com formulário para receber **peso** e **altura**. Ao enviar, o Servlet deve calcular o IMC e encaminhar os dados para uma JSP de resultado.

FÓRMULA

$$\text{IMC} = \text{peso} / (\text{altura}^2)$$

View

`formulario-imc.jsp`

Controller

`ImcServlet` via POST

Resposta

`resultado-imc.jsp` com EL/JSTL

Etapas de implementação

1

Formulário

Campos `peso` e `altura`, usando `method="post"`.

2

Servlet

Ler parâmetros com `getParameter` e converter para número.

3

Cálculo

Calcular IMC, classificar o resultado e tratar valores inválidos.

4

JSP final

Usar `request.setAttribute`, `forward` e EL para exibir.

Critérios de aceite

Fluxo correto

Formulário → Servlet → JSP, sem calcular na página.

Validação básica

Peso e altura numéricos, positivos e com mensagem de erro.

View limpa

Resultado apresentado com EL/JSTL, sem scriptlets.

LIST Atividade prática 2: cadastro com lista

Implemente uma aplicação para **cadastar usuários** e exibir a lista cadastrada, integrando formulário, **CadastroServlet**, coleção **usuarios** e uma JSP de listagem com EL/JSTL.



Artefatos que devem ser criados	
Arquivo / classe	Responsabilidade
formulario.jsp	Exibir formulário com campos <code>nome</code> , <code>email</code> e <code>curso</code> , enviando por POST.
CadastroServlet	Ler parâmetros, criar <code>Usuario</code> e adicionar à coleção e encaminhar para a listagem.
Usuario.java	Representar os dados com campos privados, construtor e getters para acesso por EL.
lista.jsp	Receber <code>usuarios</code> no request e renderizar com <code><c:forEach></code> .

Funcionamento esperado	
1	Cadastro: ao enviar o formulário, um novo objeto <code>Usuario</code> deve ser criado.
2	Coleção: manter uma lista de usuários cadastrados.
3	Encaminhamento: usar <code>request.setAttribute("usuarios", usuarios)</code> e <code>forward</code> .
4	Listagem: a JSP deve usar EL/JSTL, sem scriptlets, para exibir cada usuário cadastrado.



Desafio final: mini sistema acadêmico.

Construa um **mini sistema acadêmico** integrando os conceitos da aula: formulário, Servlet Controller, JavaBean, JSP, Expression Language e JSTL.



Entrega	O que implementar	Critério técnico
Cadastro	Formulário com matrícula, nome, e-mail e curso.	Enviar por POST para CadastroServlet e ler parâmetros com getParameter .
Confirmação	Página de retorno informando que o aluno foi cadastrado.	Usar request.setAttribute , forward e EL, como #{aluno.nome} .
Listagem	Exibir a coleção de alunos cadastrados em uma tabela.	Enviar por GET para CadastroServlet e listar alunos utilizando a classe service. Encaminhar para uma JSP renderizar a lista.



Checklist para projetos Java Web

Antes de entregar um projeto Java Web, revise o caminho completo da requisição: **mapeamento**, método, codificação, validação, despacho, view e separação MVC.

Verificações essenciais antes da entrega		
Item	Pergunta de revisão	Evidência esperada no projeto
URI mapeada	A URL acessada corresponde ao mapeamento do Servlet?	<code>@WebServlet</code> ou configuração equivalente aponta para a URI correta.
Método HTTP	A operação usa GET para consulta e POST para envio?	Formulários e links estão coerentes com a intenção da requisição.
Encoding	Caracteres acentuados chegam corretamente ao servidor e à JSP?	UTF-8 configurado antes da leitura dos parâmetros e nas páginas.
Validação	Entradas obrigatórias, numéricas ou sensíveis são verificadas?	Mensagens de erro ou sucesso são exibidas sem quebrar o fluxo.
Despacho	Foi usado forward quando a JSP precisa de atributos do request?	<code>request.setAttribute</code> ocorre antes do encaminhamento para a view.
View	A JSP apenas apresenta dados, sem regra de negócio?	Uso de EL e JSTL ; ausência de scriptlets extensos.
MVC	Controller, Model e View têm responsabilidades separadas?	Servlet controla, JavaBean/Service representa dados e JSP renderiza.

Sinais de alerta

- ! **404:** verifique contexto da aplicação e URI do Servlet.
- ! **Null:** confira nomes de parâmetros, atributos e escopos.
- ! **Acentos quebrados:** revise encoding da requisição e da página.
- ! **Atributos sumindo:** redirect cria nova requisição.
- ! **JSP pesada:** mova regra de negócio para Servlet/Model.

Referências

As fontes abaixo reúnem **especificações oficiais** e documentações de apoio para aprofundamento em Servlets, JSP/Jakarta Pages, Expression Language e JSTL/Jakarta Tags.

Especificações Jakarta EE	
1	Jakarta Servlet Specification Referência oficial para o ciclo de vida, mapeamento, request, response, filtros e despacho de requisições. https://jakarta.ee/specifications/servlet/
2	Jakarta EE Specification Guide Guia de especificações da plataforma Jakarta EE e ponto de entrada para as tecnologias usadas em Java Web. https://jakarta.ee/specifications/
3	Jakarta Pages Specification Especificação oficial de páginas JSP/Jakarta Pages, sua sintaxe, objetos implícitos e integração com componentes web. https://jakarta.ee/specifications/pages/
4	Jakarta Expression Language Referência para resolução de expressões, acesso a propriedades e uso de EL em páginas e tecnologias Jakarta EE. https://jakarta.ee/specifications/expression-language/

JSTL, tags e documentação complementar	
5	Jakarta Standard Tag Library Especificação da biblioteca padrão de tags usada para iteração, condicionais, formatação e internacionalização. https://jakarta.ee/specifications/tags/
6	Eclipse EE4J — Jakarta Tags Projeto de referência associado às tags padrão e ao ecossistema Jakarta mantido pela Eclipse Foundation. https://projects.eclipse.org/projects/ee4j.jstl
7	Oracle Java EE Tutorial — Web Tier Material histórico de apoio sobre Servlets, JSP e camada web em Java EE, útil para comparação conceitual. https://docs.oracle.com/javaee/7/tutorial/
8	Apache Tomcat Documentation Documentação do contêiner web usado com frequência para executar aplicações baseadas em Servlets e JSP. https://tomcat.apache.org/tomcat-10.1-doc/